Wasnik, Ziegler, Chitturi, Zemblaku, Dhadiala

GitHub

19 April, 2026

KNN for Movie Recommendations

The main question that we set out to answer was how effective can a KNN algorithm be used to find personalized movie recommendations.

The core of the project revolves around a dataset from the GroupLens Research Project from the University of Minnesota that contains over a hundred thousand ratings of over sixteen hundred movies. Within our code, we built a dictionary linking the users to their ratings, fitted a cosine KNN model, and created a quiz that asks a user to rate a selection of movies themselves so we can recommend them further movies.

Methods:

The movie recommendations pipeline begins with an interactive user quiz. The system fetches the 50 most popular movies from the database and presents a randomized subset of 10 for evaluation. Users rate each film from 1 to 5, or enter 0 to skip unseen movies, which are replaced dynamically from the popular pool. This produces a personalized rating vector mapping movie IDs to user preferences, which serves as input to the K-Nearest Neighbors (KNN) algorithm for generating recommendations.

We implemented scikit-learn's `nearestneighbors` model. The data is transformed using a pivot table (Figure 1) with rows representing users and columns representing movie IDs and the KNN algorithm creates a vector for each user based on their movie ratings. User similarity is determined by the angle between vectors. Values closer to 1 indicate similar preferences, 0 indicates no correlation, and -1 indicates opposite tastes. The most similar users are then used to generate movie recommendations.

User ID	Movie ID	Rating
196	242	3
186	302	3
22	377	1
186	51	2

➡

User ID	242	302	377	51
196	3	0	0	0
186	0	3	0	2
22	0	0	1	0

Figure 1: Data Transformation Example

To improve user interaction, a simple React web application with a Flask backend was created. The app presents the aforementioned user quiz function, where the user rates movies they've seen between 1 and 5, or 0 if unseen, which triggers a replacement until the application receives 10 ratings. Once complete, the results are sent as JSON to the Flask backend. The model turns the ratings JSON into a vector space which the KNN model uses to get the spaces of users with the highest cosine similarity, which is then sent back to the frontend to display.

Graph theory was used in this project to represent the results of the KNN algorithm, where one set of nodes corresponds to users and the other to the movies (Figure 2). A similarity score is used to identify the most similar user, and ten movies are displayed with predicted interest ratings for each. After identifying the closest user through KNN, the graph connects the target user to the most similar user and highlights the movies associated with that user, allowing the recommendation process to be visualized through these network relationships.

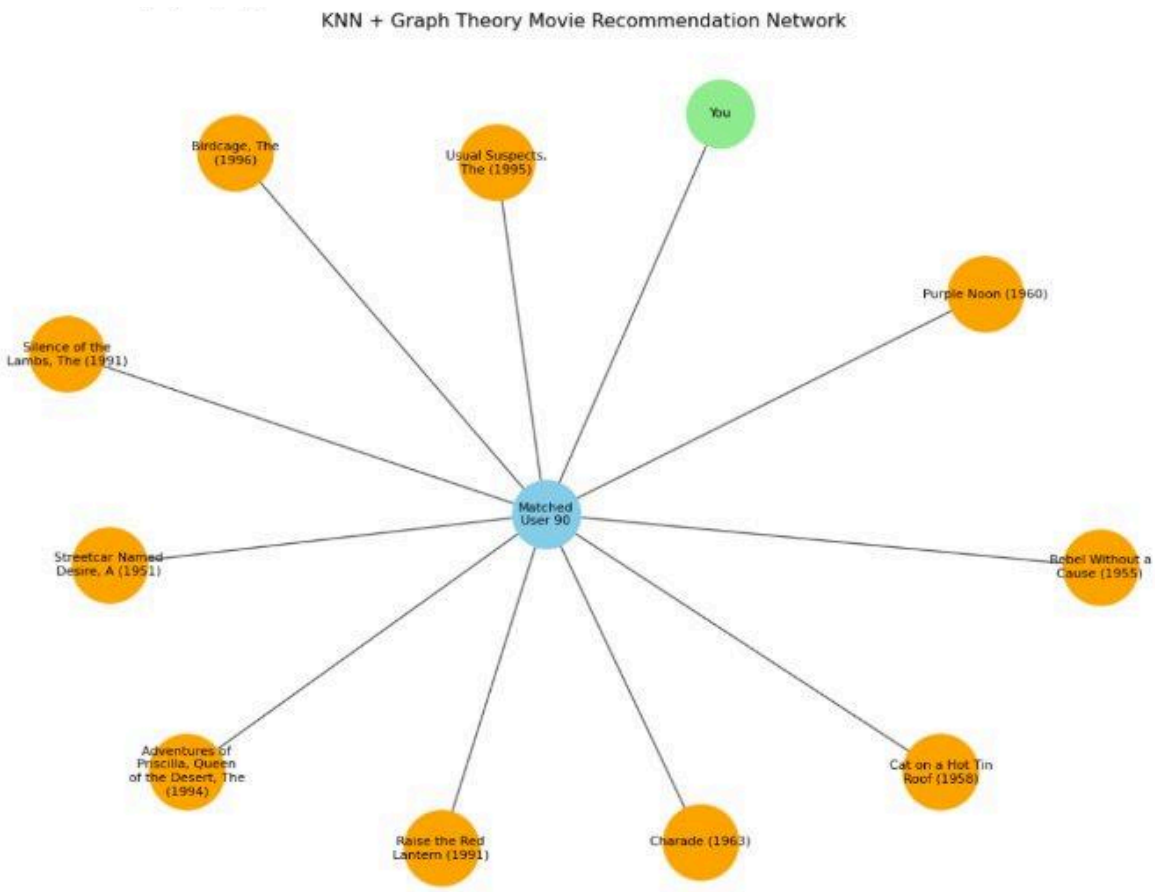

Figure 2: Graph theory bipartite graph

Results:

One of the last tasks that we wanted to follow through with was testing the efficacy of our algorithm and shifting around parameters that would result in the highest success. This would require making a simulation that would imitate the results of a user taking a quiz, rather than using real world quiz data. The idea would be to take a random user, sort the user by its highest rated movies (>= 4 stars), and then create a vector with only 10 of those movies. This would be the crux of the script, using that vector to imitate actual results from a quiz to run the Nearest Neighbors algorithm on. The results of the quiz would provide a list of recommended movies it finds, from this list, if the recommended movies matches a movie that the original user has previously watched, the quiz is then considered successful. If not, it was a failure. From this idea, it was just a matter of upscaling and repeating the process many times to find the success rate of the algorithm.

Here's an example of how one run under these parameters may work:

20 Trials/user
10 Questions on the quiz
4-Star ratings or higher
20 Recommendations outputted

Overall results of Trial 1:

Eligible Users	3283
Executed Trials	65660
Successes	38013
Failures	27647
Success Rate	57.89%

Here, we found that the simulation script was able to find a liked and watched movie over 57.89% of the trials across the datasets. Overall, these results suggest that the model has a lot of predictive ability under Nearest Neighbors, but its performance can certainly be improved. If we were to increase the number of questions that the quiz would ask, it would result in a more accurate output that we can observe that leads to a higher success rate:

20 trials/user
20 Questions on the quiz
4-Star ratings or higher
20 Recommendations outputted

Overall Results for Trial 2:

Eligible Users	2424
Executed Trials	48480
Successes	30922
Failures	17558
Success Rate	68.78%

Here, we found that the simulation script was a lot more successful overall, around 68.78% able to successfully recommend a movie the user would watch and enjoy.

Conclusion:

Our project demonstrated that a cosine similarity KNN algorithm is capable of creating personalized movie recommendations. The baseline success of our model was 57.89%, and 68.78% after increasing questions to 20. The model showed that relating user movie preference has meaningful predictive capabilities. These results confirm that KNN can effectively leverage user rating patterns to identify shared movie preferences. This can be further refined by incorporating more quiz questions or increasing our dataset to include additional similarity metrics.